

COMP2300/6300 Practice Final Exam

Student ID:

u							
---	--	--	--	--	--	--	--

Make sure you read each question carefully. Questions are not equally weighted, and the size of the answer box is not necessarily related to the length of the expected answer or the number of marks given for the question.

All answers must be written in the boxes provided in this booklet. You will be provided with scrap paper for working, but only the answers written in this booklet will be marked. Do not remove this booklet from the examination room. There is additional space at the end of the booklet in case the boxes provided are insufficient. If you use these extra pages, make sure you clearly label which question the answer refers to.

Greater marks will be awarded for short, concrete answers than long, vague/rambling ones. Marks may be deducted for providing information that is irrelevant to a question.

For examiner use

Question 2

Part 1

Your discoboard has a random number generator (RNG) which can produce a 32-bit random number (each bit has a 50% chance of being 0 or 1) in the RNG data register (RNG_DR), which is mapped to address 0x50060800.

Assuming the RNG has been initialised and is running correctly, write a program which branches to **foo** 25% of the time, branches to **bar** 75% of the time.

Question 3

Part 1

On the axes provided, draw the “trajectory” of the value in the program counter register (**pc**) over time during the execution of the following program (you may add labels or other annotations to the graph where appropriate).

```
1  delay:
2      subs r0, #1
3      bpl delay
4      bx lr
5
6  main:
7      ldr r1, =0xDEADBEEF
8      mov r0, #4
9      bl delay
10
11 end:
12     nop
13     b end
```



Part 2

What will be the values in **r0** and **r1** when the program reaches the end?

Question 4

func:

```
cmp    r1, #1
ble    .L4
adds   r3, r0, #4
add    r1, r0, r1, lsl #2
movs   r0, #0
```

.L3:

```
ldr    r2, [r3], #4
add    r0, r0, r2
cmp    r3, r1
bne    .L3
bx     lr
```

.L4:

```
movs   r0, #0
bx     lr
```

Write a short program in a programming language of your choice which might have compiled down to the above ARM assembly code (you may assume that the ARM code uses the standard ARM calling convention). Don't worry if you can't remember the exact syntax for your programming language, it's what it *does* that's important.

Question 5

Part 1

Is the SystemWorkbench IDE you've been using in this course an **optimising**¹ compiler/assembler? Explain your answer—provide short code snippets or diagrams where necessary.

¹An optimising compiler is one which is allowed to change the specific instructions & their ordering as long as the program still generates the same “result”.

Part 2

Is it *possible* to write an **optimising** compiler/assembler for ARM assembly language? Explain your answer—provide short code snippets or diagrams where necessary.

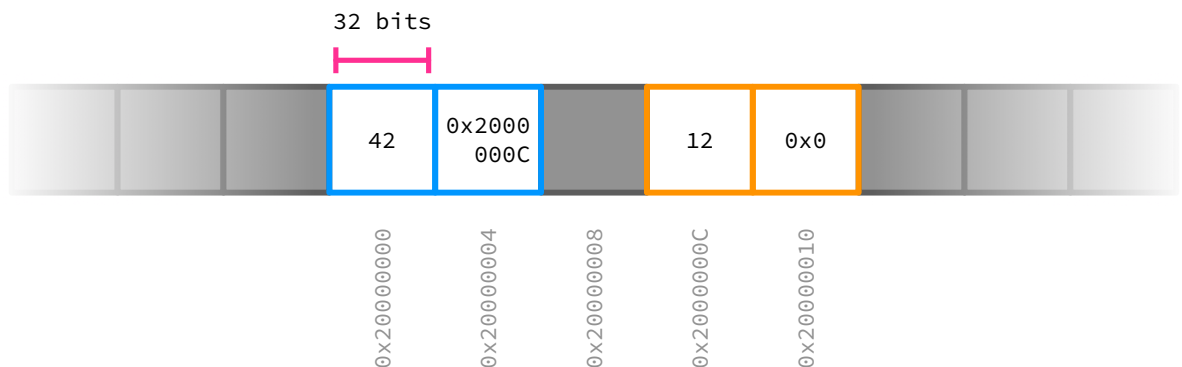
Question 6

When designing a networking protocol, what are the *essential* things that the sender and receiver must agree on? Explain your answer—be as specific as you can.

Question 7

Consider a **pair** data structure: the first word holds a 32-bit integer *value*, and the second word holds one holds an *address*. Pairs can be used to make a list: each pair holds the value in the first word and the address of the next pair in the second word, or 0 if there's no next value (i.e. the value is the last one in the list).

Here's an example: the following picture shows (in memory) a list of length 2 containing the values 42 and 12. The first pair is in blue, and the second pair is in orange. Notice how the second word in the blue pair contains the address of the first word in the orange pair, and the second word in the orange pair is 0 (so this is the last element of the list).



Part 1

Write a **recursive** function which takes the address of a pair data structure as the single argument (in **r0**) and calculates the **length** of the list.

Part 2

Is it possible to write a non-recursive version of the “length” function? If so, what are the pros and cons of the recursive vs non-recursive approach? Be as specific as you can—provide short code snippets or diagrams where necessary.

Question 8

Part 1

Is the use of a frame pointer register (a register which holds the address of the *base* of the current stack frame) **necessary** when writing an ARM assembly function? Why or why not? Explain your answer—be as specific as you can.

Part 2

If an ARM assembly program *does* use the **fp** register to store the frame pointer during all function calls, can you use this information to *profile* your program—that is, to determine which parts of the program are taking up the most CPU cycles? If so, how? If not, why not? Explain your answer—provide short code snippets or diagrams where necessary.

Question 9

If you are given an ARM assembly program to run on your discoboard which uses a rising-edge triggered interrupt on one of the GPIO pins, how many different ways could you **stop** (i.e. disable) the interrupt from triggering? List as many as you can.

Question 10

Part 1

What does it mean to say that the ARMv7 discoboard is a **32-bit** architecture? Explain your answer—be as specific as you can.

Part 2

Is it possible for a 32-bit microcontroller to have components of other “widths”, e.g. 16-bit or 64-bit registers? If not, why not? If so, what reasons might the designers have for including these components on the board? Explain your answer—provide short code snippets or diagrams where necessary.

Question 11

Part 1

Consider a multi-tasking OS for your discoboard similar to *yournameOS* from lab 11, but with the added feature that each process can be in one of three states: **ready**, **waiting**, or **terminated**. Your job is to design a process control block (PCB) data structure for this OS—draw a diagram explaining the elements of this data structure. The PCB data structure must still contain all the information required to resume the execution of a process.

Part 2

If a function only reads & writes the registers **r0–r12** but does not touch other memory (including memory-mapped configuration registers), is there a risk of data corruption or incorrect results if the function is running in a process on a multi-tasking OS? If not, why not? If so, how—and what concurrency/asynchronism techniques could you use to prevent this from happening? Be as specific as you can.

Question 12

Straight from the discoboard manual (Section 3.3.7, p88), here are two of the registers which control access to the discoboard's flash memory.

3.7.5 Flash status register (FLASH_SR)

Address offset: 0x10

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BSY
															r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPTV ERR	RD ERR	Res.	Res.	Res.	Res.	FAST ERR	MISS ERR	PGS ERR	SIZ ERR	PGA ERR	WRP ERR	PROG ERR	Res.	OP ERR	EOP
rc_w1	rc_w1					rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1		rc_w1	rc_w1

3.7.6 Flash control register (FLASH_CR)

Address offset: 0x14

Reset value: 0xC000 0000

Access: no wait state when no Flash memory operation is on going, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOCK	OPT LOCK	Res.	Res.	OBL_ LAUNCH	RD ERRIE	ERR IE	EOP IE	Res.	Res.	Res.	Res.	Res.	FSTPG	OPT STRT	STRT
rs	rs			rc_w1	rw	rw	rw						rw	rs	rs
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MER2	Res.	Res.	Res.	BKER	PNB[7:0]								MER1	PER	PG
rw				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Your job is to use the following Flash memory programming sequence (again, straight from the discoboard manual) to write the 64-bit unsigned integer value 0xAAAA BBBB CCCC DDDD to the FLASH memory storage at the address 0x08080000. For **each step**, write a sequence of ARM assembly instructions in the box provided to perform the step.

Part 1

Check that no Flash main memory operation is ongoing by checking the BSY bit in the Flash status register (FLASH_SR, mapped to 0x40022010).

Part 2

Set the PG bit in the Flash control register (FLASH_CR, mapped to 0x40022014).

Part 3

Perform the data write operation (write the 64-bit unsigned integer value 0xAAAA BBBB CCCC DDDD) to the FLASH memory storage at the address at the desired memory address (0x08080000). Only a double word can be programmed:

- Write a first word in an address aligned with double word
- Write the second word

Part 4

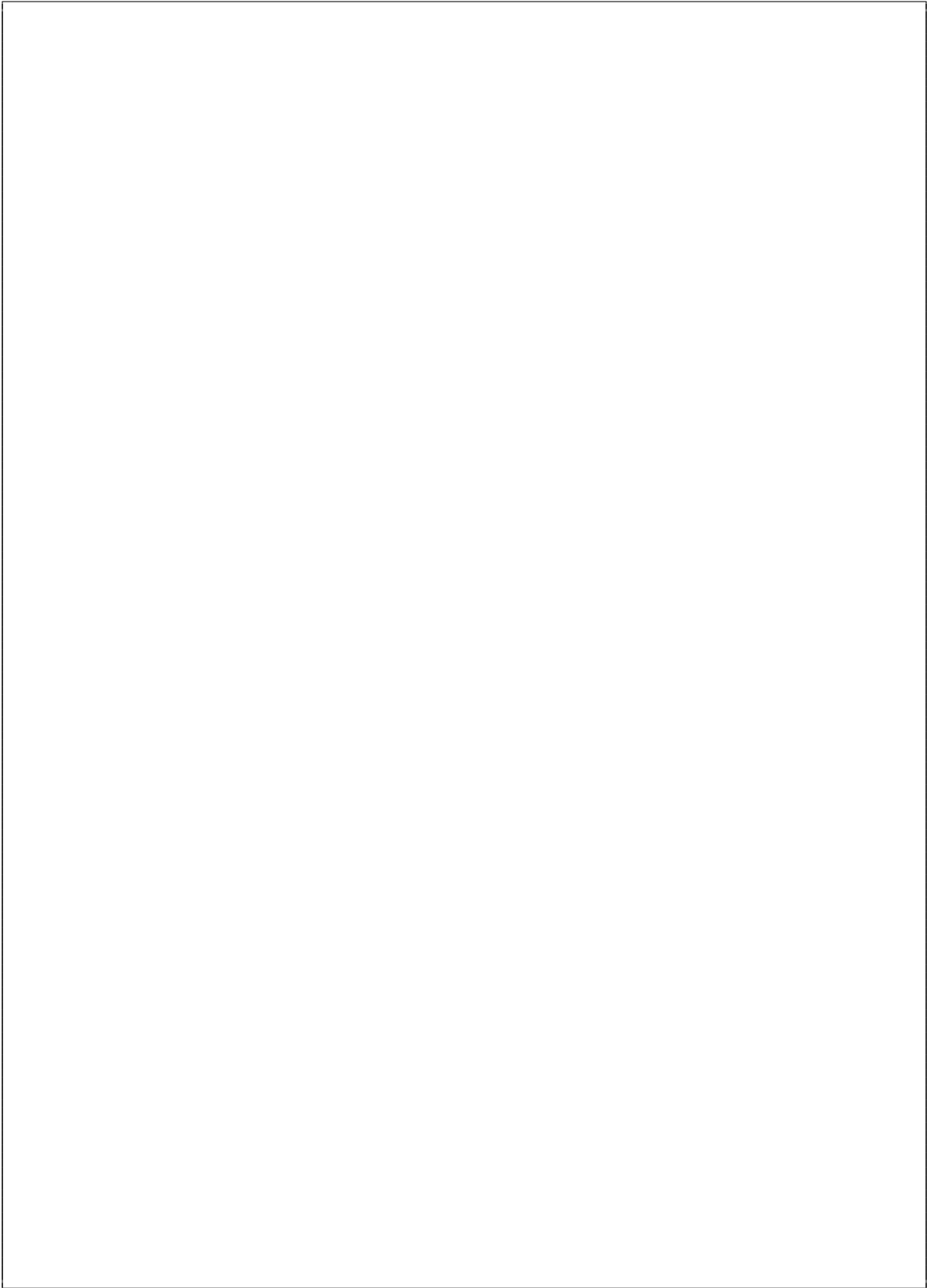
Wait until the BSY bit is cleared in the FLASH_SR register.

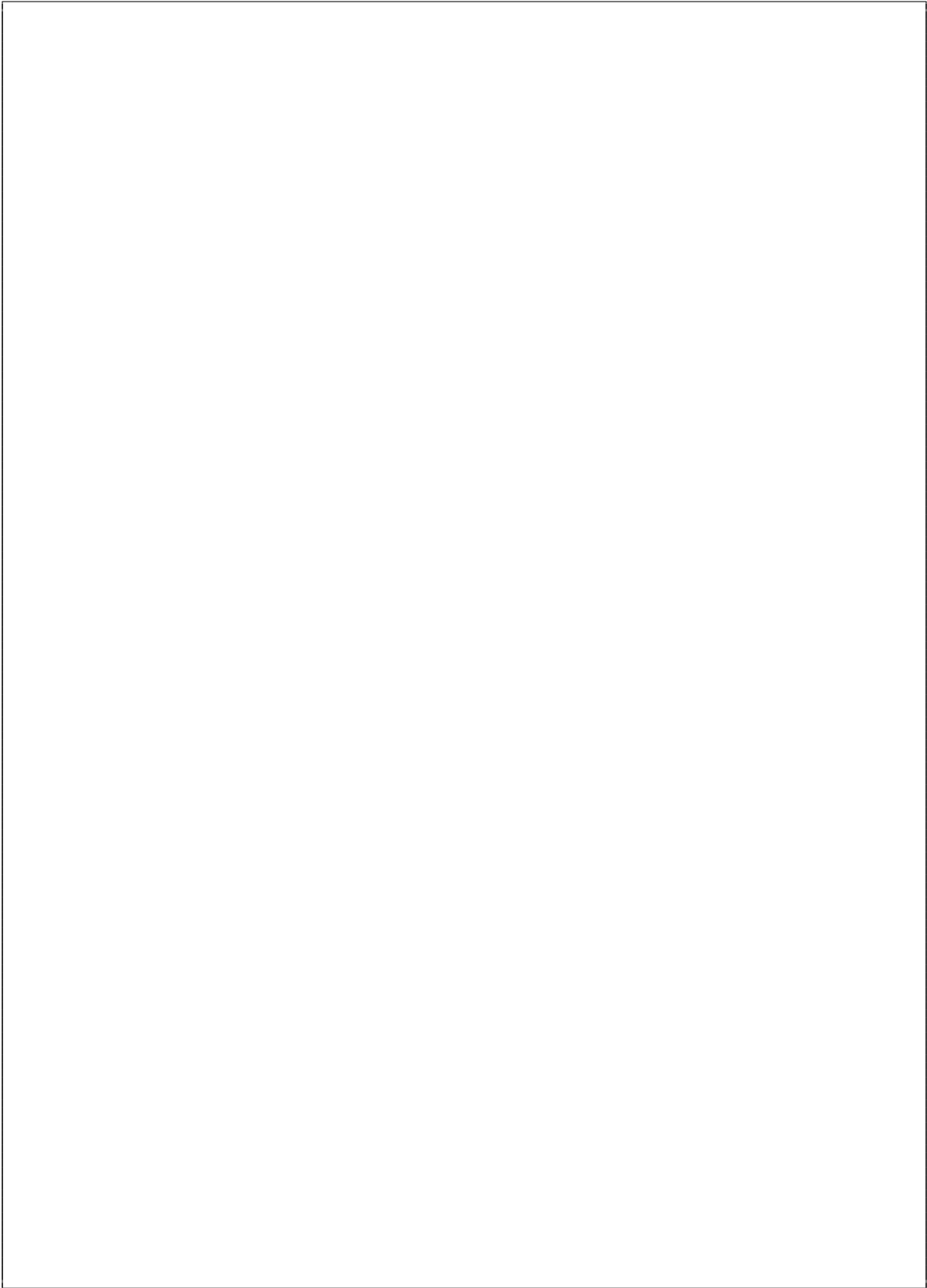
Part 5

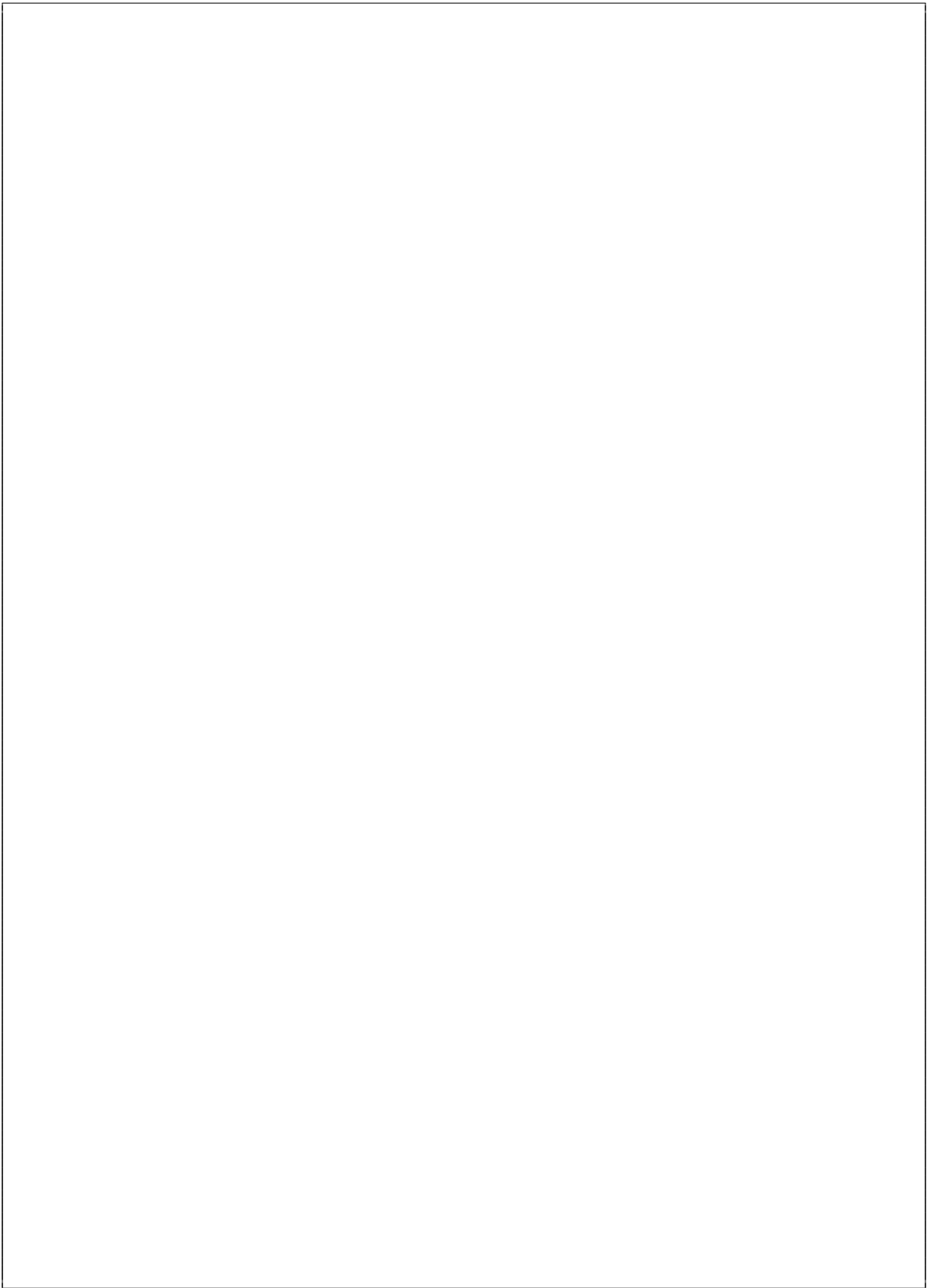
Check that EOP flag is set in the FLASH_SR register (meaning that the programming operation has succeed), and clear it with software.

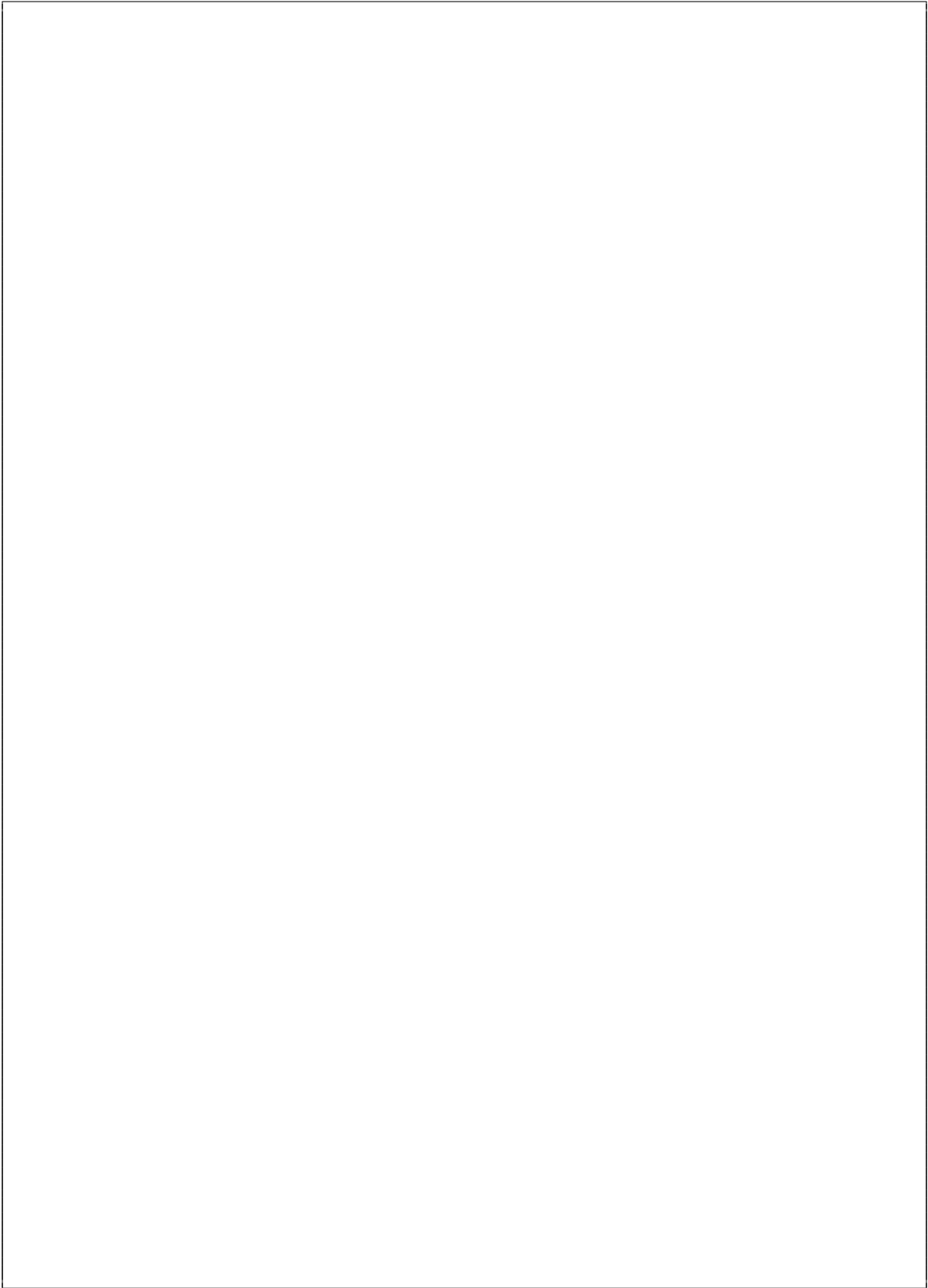
Part 6

Clear the PG bit in the FLASH_SR register if there are no more programming requests.







A large, empty rectangular box with a thin black border, occupying the central portion of the page. It is intended for students to write their answers to the exam questions.